Ticket Ninja 🥷

Software Requirements Specification

1

2/12/2026

Group 2

Jake Webb, Mark Guggenheim, Hector Estrada

Prepared for
CS 250- Introduction to Software Systems
Instructor: Dr. Gus Hanna
Fall 2024

Revision History

Date	Description	Author	Comments
2/19/26	Version 1	Group	First Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4

4. Software Design Specification 5

4.1 UML Diagram

4.1.1 UML Diagram Description

4.2 SWA Diagram

4.2.1 SWA Description

1. Introduction

This SRS document details the requirements for the Ticket Ninja ticket manager software system.

1.1 Purpose

The purpose of this SRS document is to explain in detail the vision and purpose behind Ticket Ninja. It is intended for software engineers to read and understand what is necessary to build Ticket Ninja accurately based on the design specifications.

1.2 Scope

Ticket Ninja will be used as an interface for moviegoers to browse for and purchase tickets for movies on-location or online across all theaters in the U.S. It should have the capabilities to allow users to search for specific movies at specific locations, dates, and times, pick their seat(s), choose between discounted child, senior, or student tickets, or buy the standard price adult ticket(s), preorder food and drinks for the movie, and finally acquire payment information for a 3rd party payment processor.

In order to accomplish these goals, Ticket Ninja will be able to run on several web browsers, handle at least 5,000,000 requests simultaneously, use location information to find nearby theaters, communicate with the software used to handle food/drink orders at the desired theater location, generate a functionally infinite amount of unique ticket codes, and properly charge customers for their tickets and food/drink orders via cash (on-location machines only, USD) or digital payment accepted by our linked payment processor.

1.3 Definitions, Acronyms, and Abbreviations

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.
SRS-Software Requirements Specification
UI-User interface
API- Application programming interface
DB-Database
MTBF-Mean Time Between Failures

1.4 References

We haven't used any references at the moment. All information on this document has originated from our general knowledge and vision for the system.

1.5 Overview

This document outlines the requirements for the TicketNinja system.

Section 1 introduces the purpose and scope of the system and explains how this document is organized.

Section 2 provides a general description of TicketNinja, including what the system is intended to do, who will use it, and the overall constraints and assumptions that affect development.

Section 3 contains the detailed functional and non-functional requirements that will guide the design and implementation of the system.

Section 4 includes supporting analysis models such as diagrams that help visualize system behavior.

Section 5 describes how changes to this document will be handled throughout the project.

2. General Description

This section provides an overview of the TicketNinja system and the environment in which it will operate. It explains the overall purpose of the product, the types of users who will interact with it, and the general limitations or constraints that may affect development.

This section does not list specific technical requirements. Instead, it helps give context so that the detailed requirements in the next section are easier to understand.

2.1 Product Perspective

- Theater seat management systems
- Theater food ordering systems
- Third-party payment processors.
- Location services APIs

2.2 Product Functions

-User account creation and login
-Movie search and filtering
-Seat collection and reservation
-Ticketing pricing calculation

-Food and beverage pre-ordering
-Ticket generation and validation
-Email confirmation

2.3 User Characteristics

- Primary Users:

-General public(ages 13+)
-Basic Internet literacy
-Uses mobile or desktop browsers

- Secondary Users:

-Theater staff validating tickets
-System administrators

- Assumptions:

-Theater staff validating tickets
-Users can input payment information correctly.

2.4 General Constraints

System operates with basic web browsers (Firefox, Google Chrome, Safari, etc)
Follows proper third-party payment processor policies
System must support high traffic load during new movie releases
Must integrate smoothly with theater seating and food management system

2.5 Assumptions and Dependencies

-Third party payment API remains operational
-Theater APIs provide real-time seat availability
-Users have stable internet connection
-Hosting infrastructure supports required traffic load

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

-Provide responsive layout for desktop and mobile
-Display seat maps graphically
-Prevent selection section of already-booked seats
-Display seat maps graphically
-Provide confirmation page with ticket code
-Provide accessible color contrast and keyboard navigation

3.1.2 Hardware Interfaces

-Support standard web browsers
-Support touchscreen kiosks
-Support QR code scanners for tickets validation
-Support receipt printer(kiosk only)

3.1.3 Software Interfaces

-Payment processing API (Stripe or equivalent)
-Theater seat management API
-Theater food ordering API
-Email notification service API

3.1.4 Communications Interfaces

-All communication shall use HTTPs (TLS 1.2 or higher)
-Payment communications shall comply with PCI-DSS
-API communication must use JSON format
-System shall support geolocation services

3.2 Functional Requirements

3.2.1 <Functional Requirement or Feature #1>

3.2.1.1 Introduction

-Allows users to search and filter available movies.
3.2.1.2 Inputs
-Movie title (optional)
-ZIP code or location
-Date
-Time range

3.2.1.3 Processing
-Query database for matching showtime
-Filter result

3.2.1.4 Outputs
-List of available movies and showtimes
-Theater location details

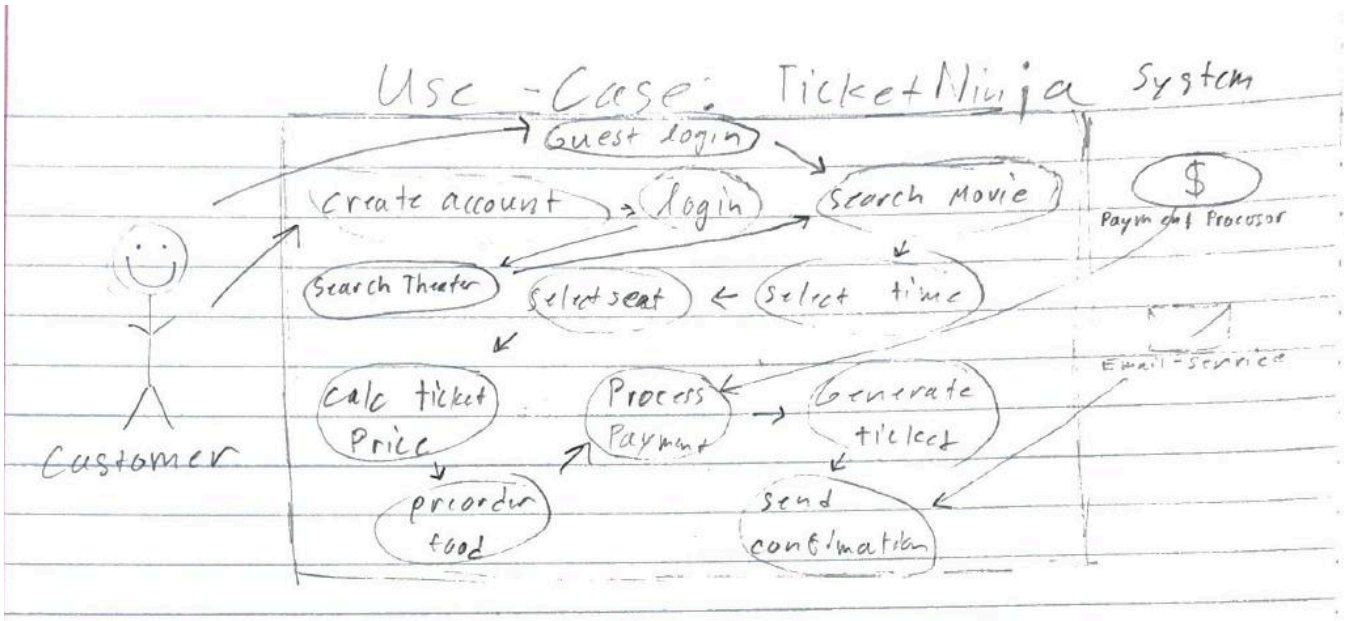
3.2.1.5 Error Handling
-Display message if no results found
-Validate location input format

3.2.2 <Functional Requirement or Feature #2>

…

3.3 Use Cases

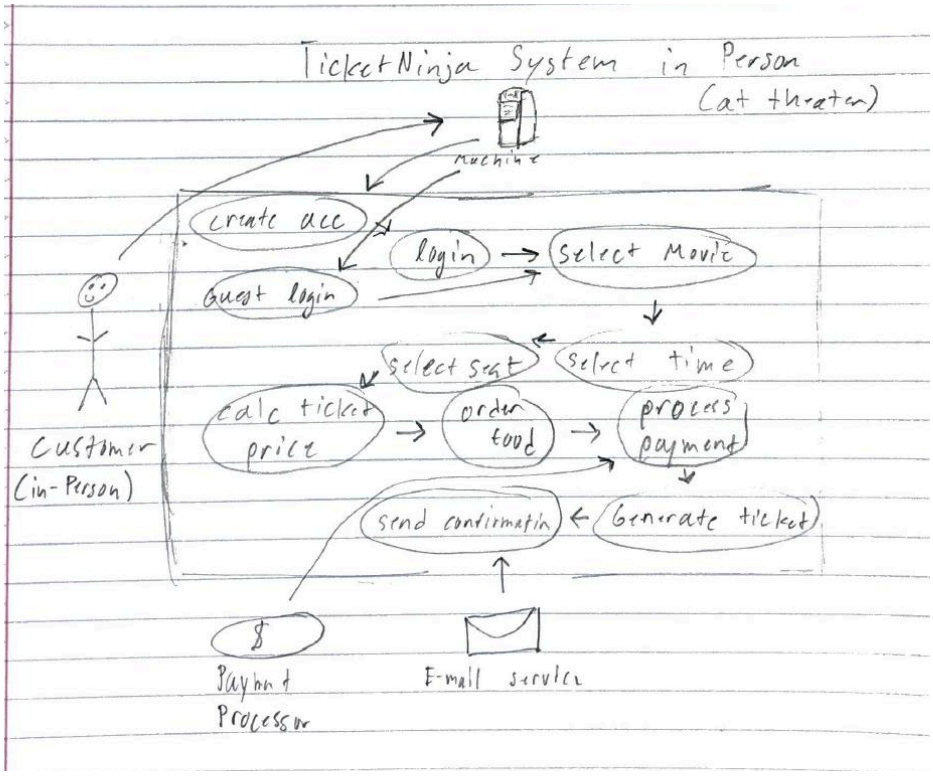
3.3.1 Use Case #1: Online Ticket Ordering



Actor: Customer
Steps:

1. User logs in/uses guest account
2. User selects location
3. User selects movie
4. User selects seas
5. User selects ticket type
6. User entering payment info
7. System validates payment
8. System generates ticket

3.3.2 Use Case #2: Ordering Tickets at a Theater



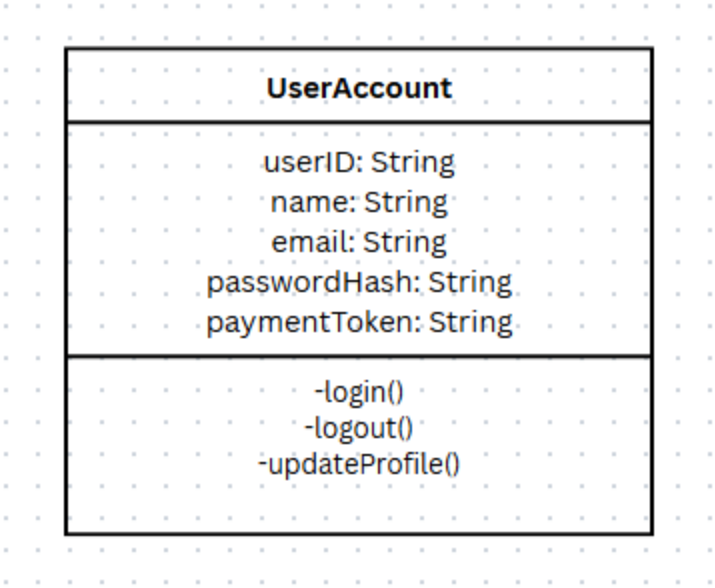
Actor: Customer
Steps:

1. User logs in/uses guest account on ticketing machine
2. User selects location
3. User selects movie
4. User selects seas
5. User selects ticket type
6. User entering payment info
7. System validates payment
8. System generates ticket and sends user an email with code/QR code

3.4 Classes / Objects

3.4.1 UserAccount

3.4.1.1 Attributes

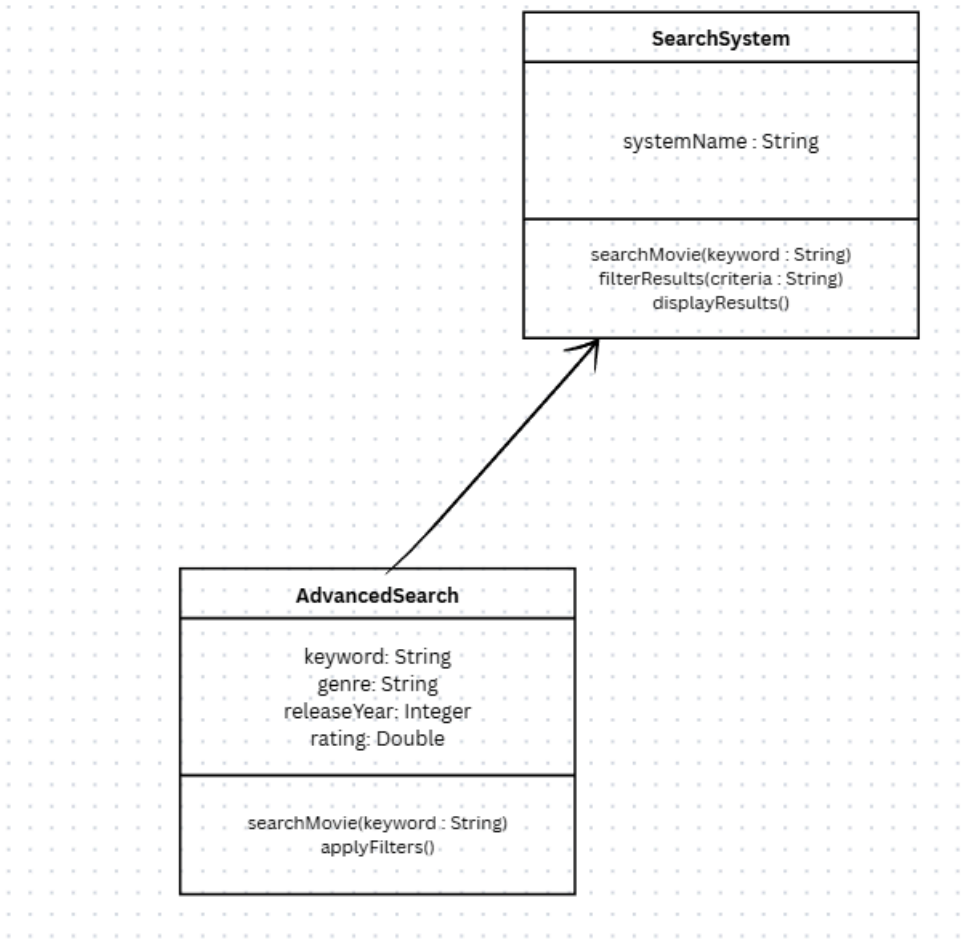


-userID
-name
-email
-passwordHash
-paymentToken

3.4.1.2 Functions
-login()
-logout()
-updateProfile()

<Reference to functional requirements and/or use cases>

3.4.2 Search System



3.4.2.1 Attributes
systemName
genre
releaseYear
rating

3.4.2.2 Functions
searchMovie(keyword)
filterResults(criteria)
applyFilters()
displayResults()

3.5 Non-Functional Requirements

3.5.1 Performance

- Must complete location handling and payment processing in less than 3 seconds

3.5.2 Reliability

- Must be able to handle 5 million concurrent requests/users

3.5.3 Availability

- Servers active 24/7, only down for maintenance, the times of which are communicated to users

3.5.4 Security

- Customer payment data should not be stored in case of breach
- System should not be prone to hacking
    - No backdoors or admin access given to theaters, all will be managed from Ticket Ninja
- Anti-scalping protection via turing test methods before reserving seats (ex. CAPTCHA)

3.5.5 Maintainability

- Service charge on all transactions large enough to sustain server expenses assuming X amount of customers per month

3.5.6 Portability

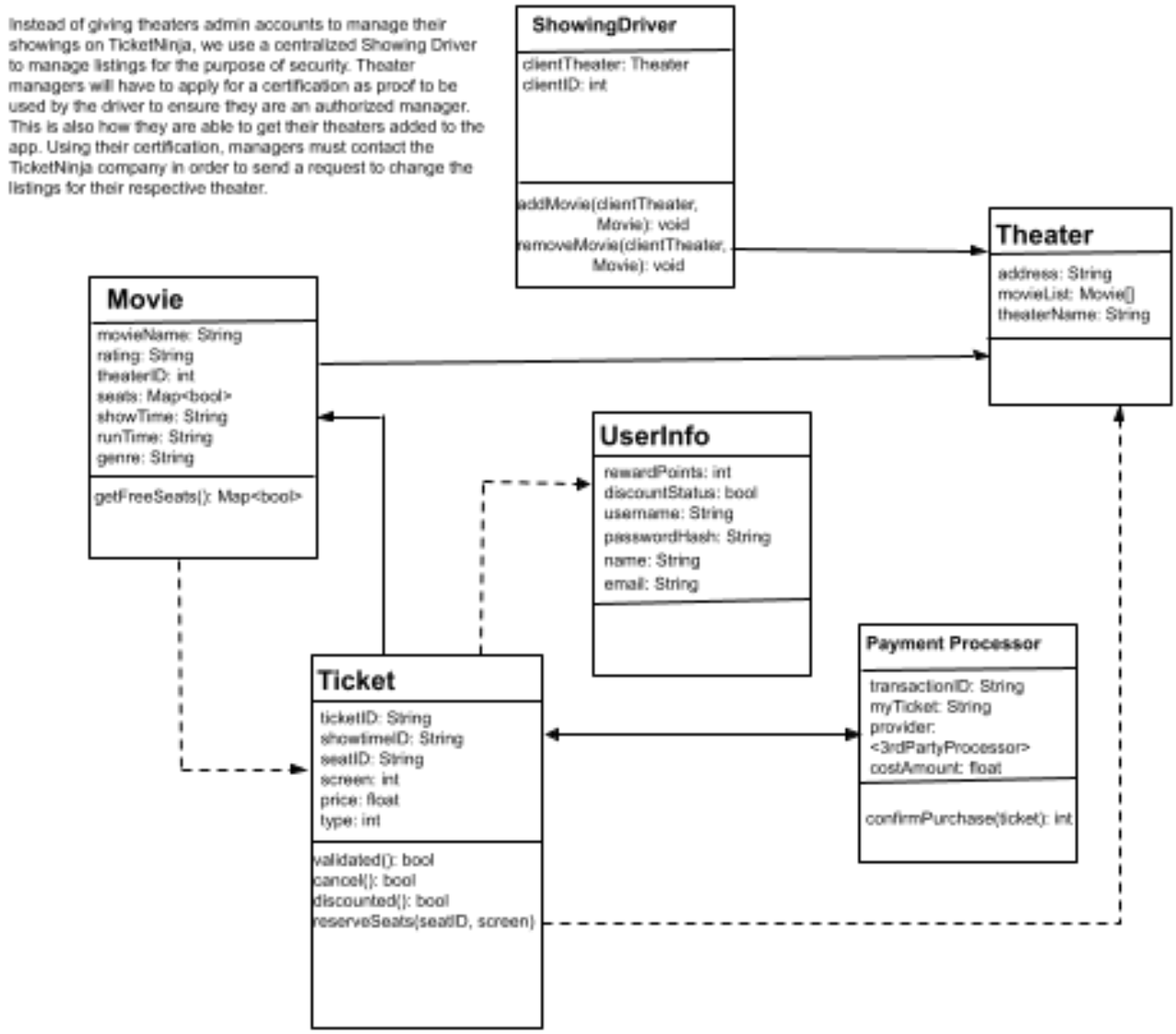
- Must be run as a web application, so that it may be used on all devices that can run an internet browser.

4. Software Design Specification

GitHub Link

4.1 UML Diagram

Figure 1: UML Diagram

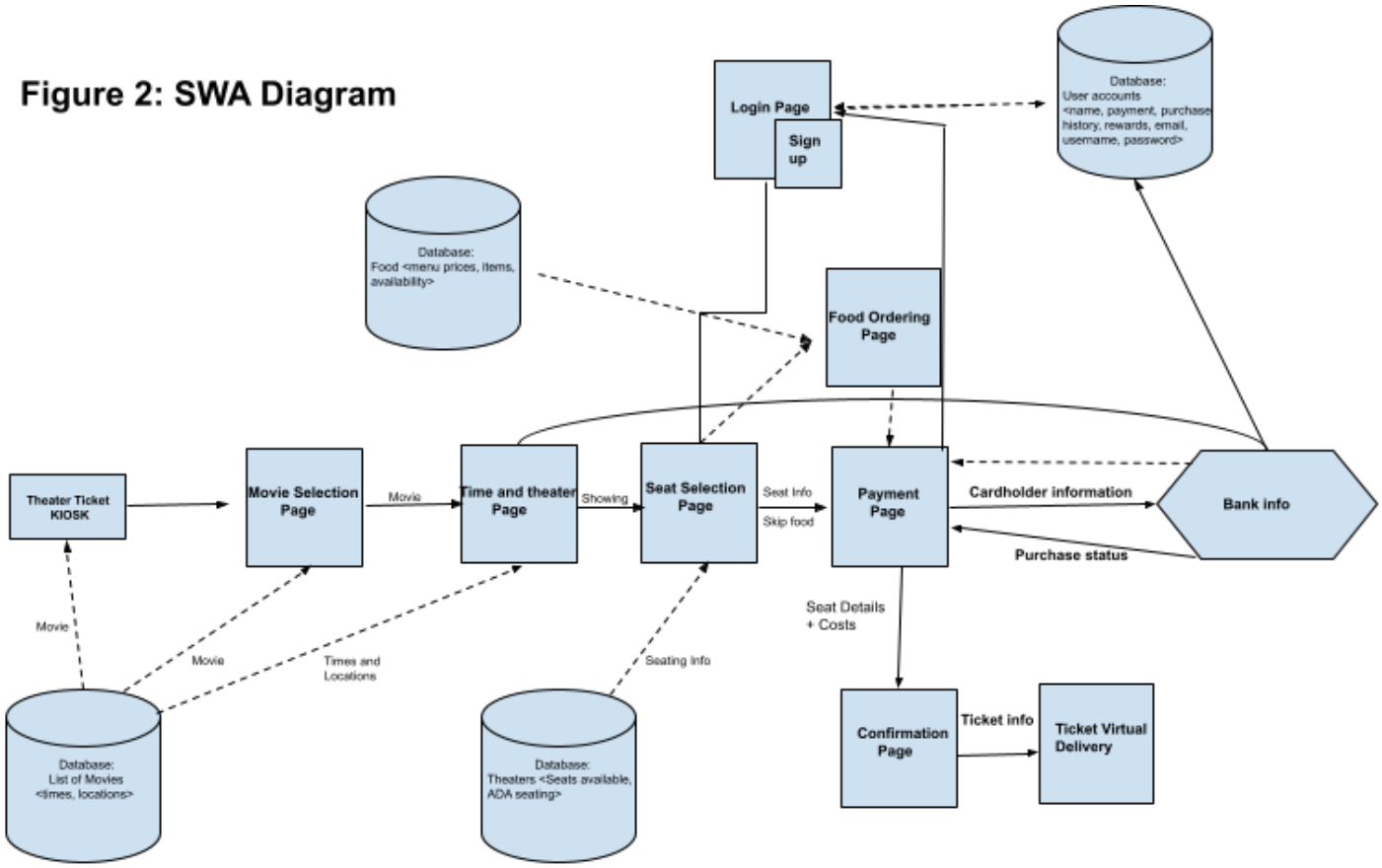


4.1.1 UML Description

This UML Diagram details how our classes will work together in order to form the TicketNinja system as a whole. We opted to not deal with location data in the sense we all know, and decided to instead allow theater managers to input all relevant information (city, address, theater name, etc.) when signing up for our service. The signing up and communication between managers and the TicketNinja system do not occur within the web application, and is managed elsewhere in the company. The main link from managers to TicketNinja is the Showing Driver, which takes in a request to edit theater showings from another system not included in the web app. The Theater

class stores all of the relevant location data and contains a list of movies playing. When users first connect to our web app, they will search for their desired theater before anything else. The Movie class contains lots of information about the movie, and keeps track of which seats in a particular screening are available. UserInfo stores account information and rewards points that are collected from using our service, and can be used to purchase tickets and food. The Ticket class stores all the information that will be sent out in an email after the purchase, and keeps track of discount status if the customer applies a discount. The Ticket class sends only the transaction data to the payment processor, which then connects with a third party payment processor system in order to properly charge the customer. This is also a security measure, since data breaches on our end will not reveal any payment information from our customers.

4.2 SWA Diagram



4.2.1 SWA Description

This SWA diagram shows the flows of information between various systems contained within TicketNinja. While it doesn't specify classes and fields like the UML diagram, the SWA more clearly shows the routes data takes between modules. For the most part, it is pretty linear starting

from left to right. We have a few databases and third party programs that interact with modules in various places like the movie selection page and food ordering page.